

Input: a+b*c-d

Output: abc*+d-

Topic: Convert Infix to Postfix Expression

Algorithm

- Take an empty string named *postfix_expression*
- For each character in the infix expression do
 - a. If the current character is an operand, then append it to the end of the *postfix_expression*
 - b. Else [the current character is an operator]
 - i. If the stack is empty or contains a left parenthesis '(' on top, push the current character onto the stack.
 - ii. If the current character is '(', push it into the stack.
 - iii. If the current character is ')', pop the stack and append the operator(s) to the end of the *postfix_expression* until the left parenthesis is found.
 - iv. If the current character has higher precedence than the top of the stack, push it onto the stack.
 - v. If the current character has lower precedence than the top of the stack, pop and appends the top operator to the end of the *postfix_expression*. Then test the current character against the new top of the stack.
 - vi. If the current character has equal precedence with the top of the stack, use the associativity rule.
 1. If the associativity is Left to Right, then pop and print the top. Then test the current character against the new top of the stack.
 2. If the associativity is Right to Left, then push the current character.
- After iterating all the characters of the infix expression, pop all the characters from the stack and append those characters to the end of the *postfix_expression* respectively.

Topic: Evaluation of Postfix Expression

Algorithm

- For each character in the postfix expression do
 1. If it is an operand then push it onto the stack
 2. Else [the character is an operator, let's call it *op*]
 - a. Pop the stack for the right-hand operand (let's call it *b*)
 - b. Pop the stack for the left-hand operand (let's call it *a*)
 - c. Apply the operator to the two operands (*result = a op b*)
 - d. Push the *result* onto the stack
- After iterating all the characters of the postfix expression, you will get the final result on the top (and only element) of the stack.

Topic: Breadth First Search

Algorithm

Initialization: Take a Queue, an array named *Visited* of size V that is initialized with zero, a variable named *Visiting*, and a vector to store the visited sequence.

Step 1: Push the source/starting node/vertex to the Queue.

Step 2: Pop a node from the Queue, and put it in the variable named *Visiting*.

Step 3: If the *Visiting* node is already visited, Go to **Step 6**.

Step 4: Push the non-visited adjacent nodes of the *Visiting* node to the Queue.

Step 5: Push the *Visiting* node to the visited sequence, and mark the *Visiting* node in the visited array as visited.

Step 6: If the Queue is not Empty, Go to **Step 2**.

Topic: Depth First Search

Algorithm

Initialization: Take a Stack, an array named *Visited* of size V that is initialized with zero, a variable named *Visiting*, and a vector to store the visited sequence.

Step 1: Push the source/starting node/vertex to the Stack.

Step 2: Pop a node from the Stack, and put it in the variable named *Visiting*.

Step 3: If the *Visiting* node is already visited, Go to **Step 6**.

Step 4: Push the non-visited adjacent nodes of the *Visiting* node to the Stack.

Step 5: Push the *Visiting* node to the visited sequence, and mark the *Visiting* node in the visited array as visited.

Step 6: If the Stack is not Empty, Go to **Step 2**.

Topic: Topological Ordering

Algorithm

Steps for finding the topological ordering of a DAG:

Step 1: Compute in-degree for each of the vertices present in the DAG and initialize the count of visited nodes as 0

Step 2: Add all vertices with in-degree equals 0 into a queue

Step 3: Remove a vertex from the queue and then

- Increment count of visited nodes by 1
- Add the node in the Topological Order
- Decrease in-degree by 1 for all its neighboring nodes
- If the in-degree of a neighboring/adjacent node is reduced to zero, then add it to the queue

Step 4: Repeat **Step 3** until the queue is empty

Step 5: If the count of visited nodes is not equal to the number of nodes in the graph then the topological sort is not possible for the given graph.